MLIR Part 0 - Installing MLIR

Stephen Diehl

Installing MLIR can be a royal pain, so here are some notes on how to do it via various methods.

From Homebrew (macOS)

The simplest way to install MLIR on macOS is through Homebrew. This will install the latest stable version of LLVM which includes MLIR tools and libraries.

```
brew install llvm@20
```

Using Apt (Ubuntu 22.04)

For Ubuntu users, the official LLVM APT repository provides pre-built packages for MLIR. This method ensures you get a properly configured build with all necessary dependencies.

```
wget -qO- https://apt.llvm.org/llvm-snapshot.gpg.key | tee /etc/apt/trusted.gpg.d/apt.llvm.org.asc
add-apt-repository "deb http://apt.llvm.org/jammy/ llvm-toolchain-jammy-20 main"
apt-get update
apt-get install -y libmlir-20-dev mlir-20-tools
```

Using `llvm.sh`

The LLVM project provides an installation script that automatically sets up the APT repository and installs LLVM/MLIR. This is a convenient method for Ubuntu/Debian systems that handles repository setup automatically.

```
bash -c "$(wget -O - https://apt.llvm.org/llvm.sh)"
```

To install a specific version of LLVM:

```
wget https://apt.llvm.org/llvm.sh
chmod +x llvm.sh
sudo ./llvm.sh 20
```

Precompiled Packages

For users who prefer to download pre-compiled Linux binaries, the LLVM project provides official release packages. These are useful when you need a specific version or want to avoid compilation time. In general these packages are best effort and may not work with your distribution / glibc setup.

1. Download the appropriate precompiled release package from the LLVM releases page. Look for files named:
    - `mlir-<version>.src.tar.xz` (e.g., `mlir-20.1.4.src.tar.xz`)
2. Extract the archive to your desired location:

```
tar xf mlir-<version>.src.tar.xz
```

3. Add the binaries to your PATH. You can either:
    - Move the extracted directory to a system location:

    ```
    sudo mv mlir-<version>.src /usr/local/mlir
    echo 'export PATH=/usr/local/mlir/bin:$PATH' >> ~/.bashrc
    ```

    - Or add the current location to your PATH:

    ```
    echo 'export PATH=/path/to/mlir-<version>.src/bin:$PATH' >> ~/.bashrc
    ```

4. Verify the installation:

```
mlir-opt --version
```

Note: While precompiled packages save build time, they may not include all optimizations that would be available in a custom build from source.

From Source (macOS)

Building from source on macOS gives you the most control over the build configuration and ensures you have the latest features. This method requires more time and disk space but provides the most flexibility.

```
brew install cmake cache ninja
git clone https://github.com/llvm/llvm-project.git
mkdir llvm-project/build
cd llvm-project/build
cmake -G Ninja ../llvm \
   -DLLVM_ENABLE_PROJECTS=mlir \
   -DLLVM_BUILD_EXAMPLES=ON \
   -DLLVM_TARGETS_TO_BUILD="Native" \
   -DCMAKE_BUILD_TYPE=Release \
   -DLLVM_ENABLE_ASSERTIONS=ON \
   -DCMAKE_C_COMPILER=clang -DCMAKE_CXX_COMPILER=clang++ \
   -DLLVM_CCACHE_BUILD=ON
cmake --build . -t mlir-opt mlir-translate mlir-transform-opt mlir-runner
cmake --build . -t install
```

From Source (Ubuntu 22.04)

Building from source on Ubuntu allows for complete customization of the MLIR build. This method is recommended for developers who need to modify MLIR or want to ensure specific optimizations are enabled.

```
sudo apt-get install clang lld
git clone https://github.com/llvm/llvm-project.git
mkdir llvm-project/build
cd llvm-project/build
cmake -G Ninja ../llvm \
   -DLLVM_ENABLE_PROJECTS=mlir \
   -DLLVM_BUILD_EXAMPLES=ON \
   -DLLVM_TARGETS_TO_BUILD="Native" \
   -DCMAKE_BUILD_TYPE=Release \
   -DLLVM_ENABLE_ASSERTIONS=ON \
   -DCMAKE_C_COMPILER=clang -DCMAKE_CXX_COMPILER=clang++ \
   -DLLVM_CCACHE_BUILD=ON
cmake --build . -t mlir-opt mlir-translate mlir-transform-opt mlir-runner
cmake --build . -t install
```

From Wheel (pip)

For Python users, pre-built wheels are available through a custom repository. This is the easiest way to get started with MLIR in Python projects.

```
pip install mlir -f https://github.com/makslevental/mlir-wheels/releases/expanded_assets/latest
```

From Wheel (Poetry)

Poetry users can install MLIR through a custom source repository. This method integrates well with Python dependency management and provides version control capabilities.

```
[tool.poetry.dependencies]
python = "^3.12"
mlir = { version = "latest", source = "mlir-wheels"}

[[tool.poetry.source]]
name = "mlir-wheels"
url = "https://github.com/makslevental/mlir-wheels/releases/expanded_assets/latest"
priority = "supplemental"
```

Then to check that it works:

```
poetry run mlir-opt --version
```

From Wheel (uv)

The uv package manager provides a modern alternative for installing MLIR wheels. This method is particularly fast and efficient for Python projects.

```
uv add mlir --index mlir=https://github.com/makslevental/mlir-wheels/releases/expanded_assets/latest
```

For uv users, you can also specify the dependencies in a `pyproject.toml` file. This approach provides a declarative way to manage your project's dependencies. Add the following to your `pyproject.toml` file:

```
[project]
dependencies = ["mlir"]

[tool.uv.sources]
mlir = { index = "mlir-wheels" }

[[tool.uv.index]]
name = "mlir-wheels"
```

```
url = "https://github.com/makslevental/mlir-wheels/releases/expanded_assets/latest"
```

Then to check that it works:

```
uv run mlir-opt --version
```

Using Conda

Conda can be used to install both MLIR and the Python bindings.

```
conda install conda-forge::mlir
```

Using Docker

For users who prefer containerized environments, pre-built Docker images are available with MLIR pre-installed that I maintain here: https://github.com/sdiehl/docker-mlir-cuda. These images come in both CUDA and non-CUDA variants, supporting both Ubuntu 22.04 and 24.04.

```
# Ubuntu 24.04 Images
# With CUDA
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir20-cuda-ubuntu24.04
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir19-cuda-ubuntu24.04

# Without CUDA
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir20-ubuntu24.04
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir19-ubuntu24.04

# Ubuntu 22.04 Images
# With CUDA
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir20-cuda-ubuntu22.04
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir19-cuda-ubuntu22.04

# Without CUDA
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir20-ubuntu22.04
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir19-ubuntu22.04
```

To run any of these images interactively:

```
docker run -it ghcr.io/sdiehl/docker-mlir-cuda:<tag> bash
```

For example, to run the CUDA-enabled MLIR 20.1.4 for Ubuntu 24.04:

```
docker run -it ghcr.io/sdiehl/docker-mlir-cuda:mlir20-cuda-ubuntu24.04 bash
```

You can also use these images as a base in your own Dockerfile:

```
FROM ghcr.io/sdiehl/docker-mlir-cuda:<tag>
```

Replace `<tag>` with your desired version (e.g., `mlir20-cuda-ubuntu24.04`). For example, to pull CUDA-enabled MLIR 20.1.4 for Ubuntu 24.04:

```
docker pull ghcr.io/sdiehl/docker-mlir-cuda:mlir20-cuda-ubuntu24.04
```

Installing MLIR Python Bindings

The MLIR Python bindings are a separate package that can be installed using your favorite Python package manager. Once you've installed the `mlir-wheels` index (as above), you can install the Python bindings with one of the following commands:

```
pip install mlir-python-bindings -f https://github.com/makslevental/mlir-wheels/releases/expanded_assets/latest
```

```
poetry add mlir-python-bindings
```

```
uv add mlir --index mlir=https://github.com/makslevental/mlir-wheels/releases/expanded_assets/latest
```

```
conda install conda-forge::mlir-python-bindings
```

If that doesn't work then you'll need to build them inside of the LLVM source tree from source with the `DMLIR_ENABLE_BINDINGS_PYTHON` flag.

```
sudo apt-get install -y \
  bash-completion \
  ca-certificates \
  ccache \
  clang \
  cmake \
  cmake-curses-gui \
  git \
  lld \
  man-db \
  ninja-build \
  pybind11-dev \
  python3 \
  python3-numpy \
  python3-pybind11 \
  python3-yaml \
  unzip \
  wget \
  xz-utils

git clone https://github.com/wjakob/nanobind && \
  cd nanobind && \
```

```
  git submodule update --init --recursive && \
  cmake \
    -G Ninja \
    -B build \
    -DCMAKE_CXX_COMPILER=clang++ \
    -DCMAKE_C_COMPILER=clang \
    -DCMAKE_INSTALL_PREFIX=$HOME/usr && \
  cmake --build build --target install

cmake llvm \
  -G Ninja \
  -B build \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DCMAKE_CXX_COMPILER=clang++ \
  -DCMAKE_C_COMPILER=clang \
  -DCMAKE_PREFIX_PATH=$HOME/usr \
  -DLLVM_BUILD_EXAMPLES=On \
  -DLLVM_TARGETS_TO_BUILD="Native" \
  -DLLVM_CCACHE_BUILD=On \
  -DLLVM_CCACHE_DIR=$HOME/ccache \
  -DLLVM_ENABLE_ASSERTIONS=On \
  -DLLVM_ENABLE_LLD=On \
  -DLLVM_ENABLE_PROJECTS="mlir;clang;clang-tools-extra" \
  -DLLVM_USE_SPLIT_DWARF=On \
  -DMLIR_ENABLE_BINDINGS_PYTHON=On \
  -DPython3_EXECUTABLE=/usr/bin/python3 \ # change this to your python executable
  -DMLIR_INCLUDE_INTEGRATION_TESTS=On

cmake --build build -t mlir-opt mlir-translate mlir-transform-opt mlir-runner
cmake --build build -t install
```

The binding are then built to:

```
build/tools/mlir/python_packages/mlir_core/mlir
```

You can then package them up with the following setup.py:

```
cd build/tools/mlir/python_packages/mlir_core
python setup.py bdist_wheel
```

And install them with:

```
pip install dist/mlir_core-*.whl
```

Or you can add them to your PYTHONPATH manually. Adjust the `$HOME` to be the place where you built the MLIR source tree.

```
export PYTHONPATH=$PYTHONPATH:$HOME/build/tools/mlir/python_packages/mlir_core
```